

Course

- POST api/v1/course: Create new course

HTTP client interface showing a POST request to `http://localhost:5000/api/v1/course`. The request body is a JSON object:

```
{
  "id": "CS101",
  "course_level_id": 1,
  "name": "Introduction to Computer Science",
  "name_vn": "Giới thiệu về Khoa học Máy tính",
  "credit_theory": 3,
  "credit_lab": 1,
}
```

Status: 201 Created Time: 60 ms Size: 246 B

id	course_level_id	name	name_vn	credit_theory	credit_lab	description
CS101	1	Introduction to Computer Science	Giới thiệu về Khoa học Máy tính	3	1	Basic concepts of computer science.
CS102	2	OOP	Lập trình hướng đối tượng	4	1	Basic concepts of OOP.
NULL	NULL	NULL	NULL	NULL	NULL	NULL

- GET api/v1/courses: List all courses

o Implement filtering and sorting for this end point

HTTP client interface showing a GET request to `http://localhost:5000/api/v1/courses`. The response body is a JSON array:

```
[{"id": "CS101", "course_level_id": 1, "name": "Introduction to Computer Science", "name_vn": "Giới thiệu về Khoa học Máy tính", "credit_theory": 3, "credit_lab": 1, "description": "Basic concepts of computer science."}, {"id": "CS102", "course_level_id": 2, "name": "OOP", "name_vn": "Lập trình hướng đối tượng", "credit_theory": 4, "credit_lab": 1, "description": "Basic concepts of OOP."}]
```

Status: 200 OK Time: 14 ms Size: 624 B

HTTP client interface showing a GET request to `http://localhost:5000/api/v1/courses?filters={"name":"OOP"}&sort={"by":"credit_theory","order":"desc"}`. The request is sent, and the response is displayed in the Body tab, showing a JSON array of course details.

Query Params:

Key	Value
filters	{"name":"OOP"}
sort	{"by":"credit_theory","order":"desc"}

Response Body (JSON):

```
1 [{"id":"CS102","course_level_id":2,"name":"OOP","name_vn":"Lập trình hướng đối tượng","credit_theory":4,"credit_lab":1,"description":"Basic concepts of OOP."}]
```

- GET `api/v1/course/{program_name}`: show detail for given course

HTTP client interface showing a GET request to `http://localhost:5000/api/v1/course/Introduction to Computer Science`. The request is sent, and the response is displayed in the Body tab, showing a JSON object with course details.

Query Params:

Key	Value
Key	Value

Response Body (JSON):

```
1 [{"id":"CS101","course_level_id":1,"name":"Introduction to Computer Science","name_vn":"Giới thiệu về Khoa học Máy tính","credit_theory":3,"credit_lab":1,"description":"Basic concepts of computer science."}]
```

- REST Client interface showing a PUT request to `http://localhost:5000/api/v1/course/CS101`. The request body is a JSON object:

```
{
  "course_level_id": 3,
  "name": "Advanced Computer Science",
  "name_vn": "Khoa học Máy tính Nâng cao",
  "credit_theory": 4,
  "credit_lab": 2,
  "description": "Advanced concepts of computer science."
}
```

The response status is **204 No Content**, Time: 39 ms, Size: 134 B.

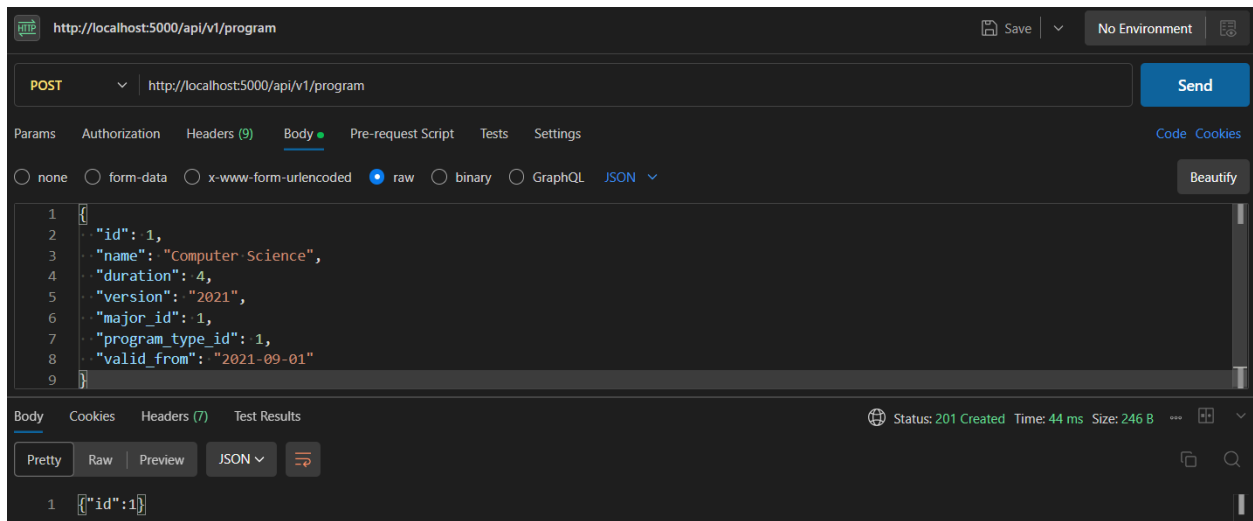
- The screenshot shows a REST client interface with the following details:

 - URL:** `http://localhost:5000/api/v1/course/CS101`
 - Method:** DELETE
 - Body:** A JSON object:

```
{  "course_level_id": 3,  "name": "Advanced Computer Science",  "name_vn": "Khoa học Máy tính Nâng cao",  "credit_theory": 4,  "credit_lab": 2,  "description": "Advanced concepts of computer science."}
```
 - Response:** Status 204 No Content, Time: 27 ms, Size: 134 B.

Program

- POST api/v1/program: Create new program



POST http://localhost:5000/api/v1/program

Body (raw):

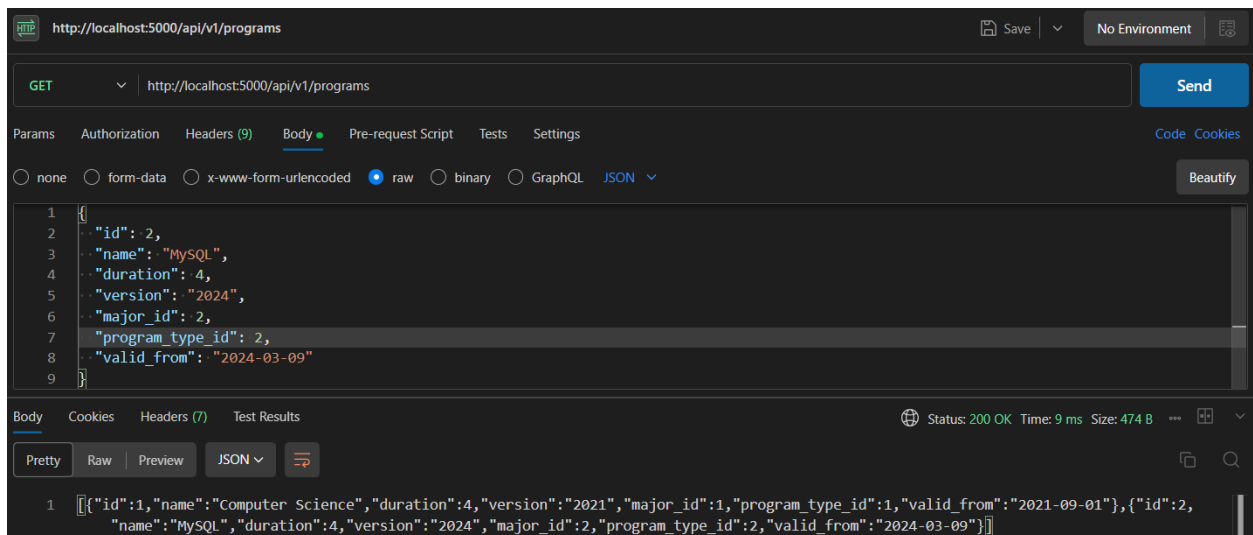
```
1 {
2   "id": 1,
3   "name": "Computer Science",
4   "duration": 4,
5   "version": "2021",
6   "major_id": 1,
7   "program_type_id": 1,
8   "valid_from": "2021-09-01"
9 }
```

Status: 201 Created Time: 44 ms Size: 246 B

id	name	duration	version	major_id	program_type_id	valid_from
1	Computer Science	4	2021	1	1	2021-09-01
2	MySQL	4	2024	2	2	2024-03-09
NULL	NULL	NULL	NULL	NULL	NULL	NULL

- GET api/v1/program: List all program

o Implement filtering and sorting for this end point



GET http://localhost:5000/api/v1/programs

Body (raw):

```
1 [
2   {
3     "id": 2,
4     "name": "MySQL",
5     "duration": 4,
6     "version": "2024",
7     "major_id": 2,
8     "program_type_id": 2,
9     "valid_from": "2024-03-09"
10  },
11  {
12    "id": 1,
13    "name": "Computer Science",
14    "duration": 4,
15    "version": "2021",
16    "major_id": 1,
17    "program_type_id": 1,
18    "valid_from": "2021-09-01"
19  }
20 ]
```

Status: 200 OK Time: 9 ms Size: 474 B

http://localhost:5000/api/v1/programs?filters={"name":"Science"}&sort={"by":"duration","order":"desc"}

GET http://localhost:5000/api/v1/programs?filters={"name":"Science"}&sort={"by":"duration","order":"desc"} Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Code Cookies

Query Params

Key	Value
filters	{"name":"Science"}
sort	{"by":"duration","order":"desc"}

Body Cookies Headers (7) Test Results Status: 200 OK Time: 24 ms Size: 361 B

Pretty Raw Preview JSON

```
1 [{"id":1,"name":"Computer Science","duration":4,"version":"2021","major_id":1,"program_type_id":1,"valid_from":"2021-09-01"}]
```

- GET api/v1/ program/{program_name}: show detail for give program

http://localhost:5000/api/v1/program/Computer Science

GET http://localhost:5000/api/v1/program/Computer Science Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Code Cookies

Query Params

Key	Value
Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK Time: 9 ms Size: 359 B

Pretty Raw Preview JSON

```
1 [{"id":1,"name":"Computer Science","duration":4,"version":"2021","major_id":1,"program_type_id":1,"valid_from":"2021-09-01"}]
```

- PUT api/v1/ program /{id}: update an existing program give course id

http://localhost:5000/api/v1/program/2

PUT http://localhost:5000/api/v1/program/2 Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Code Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "id": 2,
3   "name": "MySQL2",
4   "duration": 3,
5   "version": "2025",
6   "major_id": 2,
7   "program_type_id": 2,
8   "valid_from": "2025-03-09"
9 }
```

id	name	duration	version	major_id	program_type_id	valid_from
1	Computer Science	4	2021	1	1	2021-09-01
2	MySQL2	3	2025	2	2	2025-03-09
NULL	NULL	NULL	NULL	NULL	NULL	NULL

- DELETE api/v1/ program /{id}:delete an existing program

API Client Interface: DELETE http://localhost:5000/api/v1/program/2

Params: Authorization, Headers (9), Body, Pre-request Script, Tests, Settings

Query Params

Key	Value
Key	Value

Status: 204 No Content Time: 22 ms Size: 134 B

id	name	duration	version	major_id	program_type_id	valid_from
1	Computer Science	4	2021	1	1	2021-09-01
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Course_program

- POST api/v1/course_program: Create new course_program (Given course_name and program_name)

API Client Interface: POST http://localhost:5000/api/v1/course_program

Params: Authorization, Headers (9), Body, Pre-request Script, Tests, Settings

Body: raw

```
{
  "course_id": "CS101",
  "program_id": 1,
  "course_code": "CS101-01",
  "course_type_id": 1
}
```

Status: 201 Created Time: 35 ms Size: 246 B

course_id	program_id	course_code	course_type_id
CS101	1	CS101-01	1
CS102	2	CS102-02	2
NULL	NULL	NULL	NULL

- GET api/v1/ course_program: List all courses of given program

o Implement filtering and sorting for this end point

Postman interface showing a GET request to `http://localhost:5000/api/v1/course_program`. The response status is 200 OK. The response body is a JSON array of two course objects:

```
1 [{"course_id": "CS101", "program_id": 1, "course_code": "CS101-01", "course_type_id": 1, "course_name": "Introduction to Computer Science", "program_name": "Computer Science"}, {"course_id": "CS102", "program_id": 2, "course_code": "CS102-02", "course_type_id": 2, "course_name": "OOP", "program_name": "MySQL2"}]
```

Postman interface showing a GET request to `http://localhost:5000/api/v1/course_program?filters={'program_name':'Computer Science'}&sort={'by':'course_code','order':'asc'}`. The response status is 200 OK. The response body is a JSON array of one course object:

```
1 [{"course_id": "CS101", "program_id": 1, "course_code": "CS101-01", "course_type_id": 1, "course_name": "Introduction to Computer Science", "program_name": "Computer Science"}]
```

- DELETE api/v1/ course_program /{course_name}/{program_name}:delete an existing course_program given {course_id}/{program_id}

Postman interface showing a DELETE request to `http://localhost:5000/api/v1/course_program/CS102/2`. The response status is 204 No Content.

course_id	program_id	course_code	course_type_id
CS101	1	CS101-01	1
NULL	NULL	NULL	NULL

DB:

index.js:

```
import { createPool } from 'mysql2/promise';
```

```
import { config } from 'dotenv';
```

```
config();
```

```
const pool = createPool({  
  user: process.env.MYSQL_USER,  
  password: process.env.MYSQL_PASSWORD,  
  host: process.env.MYSQL_HOST,  
  database: process.env.MYSQL_DATABASE_NAME,  
  port: process.env.MYSQL_PORT,  
});
```

```
const connectToDatabase = async () => {  
  try {  
    const connection = await pool.getConnection();  
    console.log('MySQL Connection Successful');  
    connection.release(); // Release connection after checking  
  } catch (error) {  
    console.error('Database Connection Error', error);  
    throw error; // Throw error to handle it outside  
  }  
};
```

```
export { connectToDatabase, pool };
```

handlers:

courseHandlers.js:

```
import { createCourse, getAllCourses, getCourseByName, updateCourse, deleteCourse } from
'../models/courseModel.js';
```

```
export const createCourseHandler = async (req, res) => {
  const courseId = await createCourse(req.body);
  res.status(201).json({ id: courseId });
};
```

```
export const getAllCoursesHandler = async (req, res) => {
  const filters = req.query.filters ? JSON.parse(req.query.filters) : null;
  const sort = req.query.sort ? JSON.parse(req.query.sort) : null;
  const courses = await getAllCourses(filters, sort);
  res.json(courses);
};
```

```
export const getCourseByNameHandler = async (req, res) => {
  const course = await getCourseByName(req.params.name);
  res.json(course);
};
```

```
export const updateCourseHandler = async (req, res) => {
  await updateCourse(req.params.id, req.body);
  res.status(204).send();
};
```

```
export const deleteCourseHandler = async (req, res) => {
  await deleteCourse(req.params.id);
};
```

```
    res.status(204).send();  
  };
```

courseProgramHandlers.js:

```
import { createCourseProgram, getCoursesByProgramName, deleteCourseProgram } from  
'../models/courseProgramModel.js';
```

```
export const createCourseProgramHandler = async (req, res) => {  
  const courseProgramId = await createCourseProgram(req.body);  
  res.status(201).json({ id: courseProgramId });  
};
```

```
export const getCoursesByProgramNameHandler = async (req, res) => {  
  const filters = req.query.filters ? JSON.parse(req.query.filters) : null;  
  const sort = req.query.sort ? JSON.parse(req.query.sort) : null;  
  const courses = await getCoursesByProgramName(filters, sort);  
  res.json(courses);  
};
```

```
export const deleteCourseProgramHandler = async (req, res) => {  
  await deleteCourseProgram(req.params.course_id, req.params.program_id);  
  res.status(204).send();  
};
```

programHandlers.js:

```
import { createProgram, getAllPrograms, getProgramByName, updateProgram, deleteProgram }  
from '../models/programModel.js';
```

```
export const createProgramHandler = async (req, res) => {
```

```
    const programId = await createProgram(req.body);  
    res.status(201).json({ id: programId });  
  };
```

```
export const getAllProgramsHandler = async (req, res) => {  
  const filters = req.query.filters ? JSON.parse(req.query.filters) : null;  
  const sort = req.query.sort ? JSON.parse(req.query.sort) : null;  
  const programs = await getAllPrograms(filters, sort);  
  res.json(programs);  
};
```

```
export const getProgramByNameHandler = async (req, res) => {  
  const program = await getProgramByName(req.params.name);  
  res.json(program);  
};
```

```
export const updateProgramHandler = async (req, res) => {  
  await updateProgram(req.params.id, req.body);  
  res.status(204).send();  
};
```

```
export const deleteProgramHandler = async (req, res) => {  
  await deleteProgram(req.params.id);  
  res.status(204).send();  
};
```

models:

courseModel.js:

```
import { pool } from '../DB/index.js';
```

```
export const createCourse = async (course) => {  
  const [result] = await pool.query('INSERT INTO Course SET ?', course);  
  return result.insertId;  
};
```

```
export const getAllCourses = async (filters, sort) => {  
  let query = 'SELECT * FROM Course';  
  const queryParams = [];
```

```
  // Apply filters
```

```
  if (filters) {  
    const filterConditions = [];  
    if (filters.name) {  
      filterConditions.push('name LIKE ?');  
      queryParams.push(`%${filters.name}%`);  
    }  
    if (filters.course_level_id) {  
      filterConditions.push('course_level_id = ?');  
      queryParams.push(filters.course_level_id);  
    }  
    if (filterConditions.length > 0) {  
      query += ' WHERE ' + filterConditions.join(' AND ');  
    }  
  }  
}
```

```

// Apply sorting
if (sort) {
  const sortConditions = [];
  if (sort.by) {
    sortConditions.push(`${sort.by} ${sort.order === 'desc' ? 'DESC' : 'ASC'}`);
  }
  if (sortConditions.length > 0) {
    query += ' ORDER BY ' + sortConditions.join(', ');
  }
}

const [rows] = await pool.query(query, queryParams);
return rows;
};

export const getCourseByName = async (name) => {
  const [rows] = await pool.query('SELECT * FROM Course WHERE name = ?', [name]);
  return rows[0];
};

export const updateCourse = async (id, course) => {
  await pool.query('UPDATE Course SET ? WHERE id = ?', [course, id]);
};

export const deleteCourse = async (id) => {
  await pool.query('DELETE FROM Course WHERE id = ?', [id]);
};

```

courseProgramModel.js:

```
import { pool } from '../DB/index.js';
```

```
export const createCourseProgram = async (courseProgram) => {  
  const [result] = await pool.query('INSERT INTO Course_Program SET ?', courseProgram);  
  return result.insertId;  
};
```

```
export const getCoursesByProgramName = async (filters, sort) => {  
  let query = `  
    SELECT cp.*, c.name as course_name, p.name as program_name  
    FROM Course_Program cp  
    JOIN Course c ON cp.course_id = c.id  
    JOIN Program p ON cp.program_id = p.id  
    WHERE 1=1  
  `;  
  const queryParams = [];
```

```
// Apply filters
```

```
if (filters) {  
  if (filters.program_name) {  
    query += ' AND p.name LIKE ?';  
    queryParams.push(`%${filters.program_name}%`);  
  }  
  if (filters.course_name) {  
    query += ' AND c.name LIKE ?';  
    queryParams.push(`%${filters.course_name}%`);  
  }  
}
```

```

    if (filters.course_type_id) {
      query += ' AND cp.course_type_id = ?';
      queryParams.push(filters.course_type_id);
    }
  }
}

```

// Apply sorting

```

if (sort) {
  const sortConditions = [];
  if (sort.by) {
    sortConditions.push(`${sort.by} ${sort.order === 'desc' ? 'DESC' : 'ASC'}`);
  }
  if (sortConditions.length > 0) {
    query += ' ORDER BY ' + sortConditions.join(', ');
  }
}

```

```

const [rows] = await pool.query(query, queryParams);
return rows;
};

```

```

export const deleteCourseProgram = async (courseId, programId) => {
  await pool.query('DELETE FROM Course_Program WHERE course_id = ? AND program_id = ?', [courseId, programId]);
};

```

programModel.js:

```

import { pool } from '../DB/index.js';

```

```
export const createProgram = async (program) => {  
  const [result] = await pool.query('INSERT INTO Program SET ?', program);  
  return result.insertId;  
};
```

```
export const getAllPrograms = async (filters, sort) => {  
  let query = 'SELECT * FROM Program';  
  const queryParams = [];
```

```
  // Apply filters
```

```
  if (filters) {  
    const filterConditions = [];  
    if (filters.name) {  
      filterConditions.push('name LIKE ?');  
      queryParams.push(`%${filters.name}%`);  
    }  
    if (filters.major_id) {  
      filterConditions.push('major_id = ?');  
      queryParams.push(filters.major_id);  
    }  
    if (filterConditions.length > 0) {  
      query += ' WHERE ' + filterConditions.join(' AND ');  
    }  
  }
```

```
  // Apply sorting
```

```
  if (sort) {
```

```

const sortConditions = [];
if (sort.by) {
  sortConditions.push(`${sort.by} ${sort.order === 'desc' ? 'DESC' : 'ASC'}`);
}
if (sortConditions.length > 0) {
  query += ' ORDER BY ' + sortConditions.join(', ');
}
}

const [rows] = await pool.query(query, queryParams);
return rows;
};

export const getProgramByName = async (name) => {
  const [rows] = await pool.query('SELECT * FROM Program WHERE name = ?', [name]);
  return rows[0];
};

export const updateProgram = async (id, program) => {
  await pool.query('UPDATE Program SET ? WHERE id = ?', [program, id]);
};

export const deleteProgram = async (id) => {
  await pool.query('DELETE FROM Program WHERE id = ?', [id]);
};

```

routes:

courseProgramRoutes.js:

```
import { Router } from 'express';
```

```
import {
```

```
  createCourseProgramHandler,
```

```
  getCoursesByProgramNameHandler,
```

```
  deleteCourseProgramHandler
```

```
} from '../handlers/courseProgramHandlers.js';
```

```
const router = Router();
```

```
router.post('/course_program', createCourseProgramHandler);
```

```
router.get('/course_program', getCoursesByProgramNameHandler);
```

```
router.delete('/course_program/:course_id/:program_id', deleteCourseProgramHandler);
```

```
export default router;
```

courseRoutes.js:

```
import { Router } from 'express';
```

```
import {
```

```
  createCourseHandler,
```

```
  getAllCoursesHandler,
```

```
  getCourseByNameHandler,
```

```
  updateCourseHandler,
```

```
  deleteCourseHandler
```

```
} from '../handlers/courseHandlers.js';
```

```
const router = Router();

router.post('/course', createCourseHandler);
router.get('/courses', getAllCoursesHandler);
router.get('/course/:name', getCourseByNameHandler);
router.put('/course/:id', updateCourseHandler);
router.delete('/course/:id', deleteCourseHandler);

export default router;
```

programRoutes.js:

```
import { Router } from 'express';
import {
  createProgramHandler,
  getAllProgramsHandler,
  getProgramByNameHandler,
  updateProgramHandler,
  deleteProgramHandler
} from '../handlers/programHandlers.js';

const router = Router();

router.post('/program', createProgramHandler);
router.get('/programs', getAllProgramsHandler);
router.get('/program/:name', getProgramByNameHandler);
router.put('/program/:id', updateProgramHandler);
router.delete('/program/:id', deleteProgramHandler);
```

```
export default router;
```

Main server:

index.js:

```
import express from 'express';
```

```
import courseRoutes from './routes/courseRoutes.js';
```

```
import programRoutes from './routes/programRoutes.js';
```

```
import courseProgramRoutes from './routes/courseProgramRoutes.js';
```

```
import { connectToDatabase } from './DB/index.js';
```

```
const app = express();
```

```
// Middlewares
```

```
app.use(express.json());
```

```
app.use('/api/v1', courseRoutes);
```

```
app.use('/api/v1', programRoutes);
```

```
app.use('/api/v1', courseProgramRoutes);
```

```
const PORT = process.env.PORT || 5000;
```

```
connectToDatabase()
```

```
.then(() => {
```

```
  app.listen(PORT, () => console.log(`Server Open At port: ${PORT}`));
```

```
})
```

```
.catch((err) => {
```

```
  console.error('Error occurred with MySQL connection. Error = ', err);
```

```
  process.exit(0);
```

});